Contents

Software Development 2 Review 1 – 20%

GRIFFITH COLLEGE DUBLIN

Student name:	Student 1: Diana Ali Silva Student 2: Carlos David Urra Cabello Student 3: Georgii Taisaev Student 4: Muslim Muradov				
Student number:	Student 1: 3123965 Student 2: 3125350 Student 3: 3119655 Student 4: 3128794				
Faculty:	Computing Science				
Course:	BSCH/BSCO/EXCH		Stage/year:	2	
Subject:	Software Development 2				
Study Mode:	Full time	☑		Part-time	
Lecturer Name:	Gemma Deery				
Assignment Title:	Review 2				
Date due:	18 April 2025				
Date submitted:	18 April 2025				

Plagiarism disclaimer:

I understand that plagiarism is a serious offence and have read and understood the college policy on plagiarism. I also understand that I may receive a mark of zero if I have not identified and properly attributed sources which have been used, referred to, or have in any way influenced the preparation of this assignment, or if I have knowingly allowed others to plagiarise my work in this way.

I hereby certify that this assignment is my own work, based on my personal study and/or research, and that I have acknowledged all material and sources used in its preparation. I also certify that the assignment has not previously been submitted for assessment and that I have not copied in part or whole or otherwise plagiarised the work of anyone else, including other students.

Signed: _____ **Date:** _____

Please note: Students **MUST** retain a hard / soft copy of **ALL** assignments as well as a receipt issued and signed by a member of Faculty as proof of submission.

Project Description

The 2D Multiplayer Maze Game is a Java application inspired by the classic arcade games *Amazing Maze (1976)* and *Maze Craze (1980)*. The game challenges two players to race through a randomly generated maze to reach the exit first. Each maze is built using a recursive backtracking algorithm to ensure it is always solvable and offers a fresh layout in every round.[1,2]

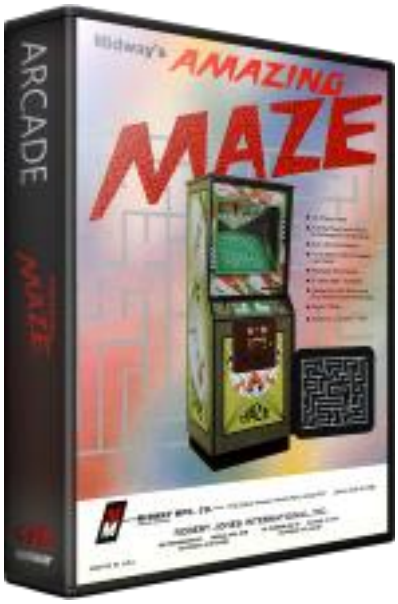


Figure1.*Amazing Maze (1976).*

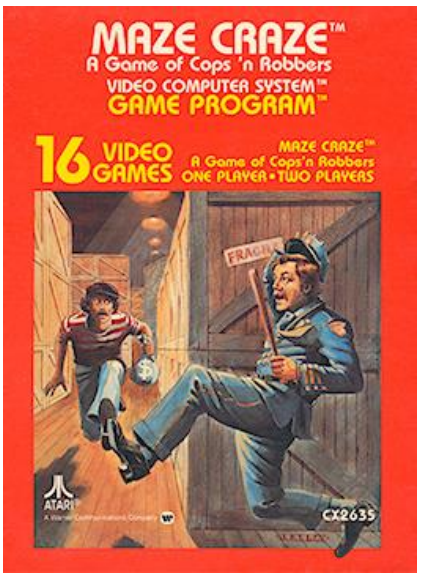


Figure 2. *Maze Craze (1980).*

Originally developed as a console-based game using ASCII characters, the project has now moved into a second iteration focused on transitioning to a graphical version using Java Swing. This phase includes the initial integration of a GUI, adaptation of local multiplayer for visual interaction, and enhancements to sprite behavior and maze fairness. Features from separate branches are being merged, and unit testing has been introduced to support continued development.[4]

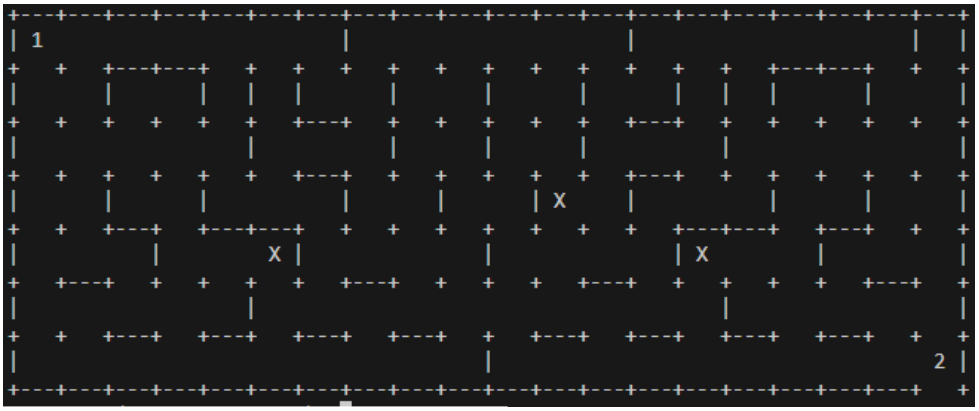


Figure 3. Maze displaying multiplayer

Made by Carlos Urra

Iteration 1 – Initial Implementation

In the first stage of the project, the team focused on building the foundation for the game. We developed core features, established a basic version of the maze, and prepared the structure for future improvements like the graphical interface and multiplayer support.

Carlos set up a GitHub repository to help the team collaborate and manage versions more efficiently. He implemented the maze generator using recursive backtracking, added the basic single-player logic including exit detection, and organized the project folders based on the structure provided in the Moodle example.[3]

Diana implemented a Java Swing window that opened when running the class. At this point, the window displayed only a white screen, but it marked the beginning of transitioning from the console-based version to a graphical user interface.[4]

Georgii added placeholder sprites, represented as 'X' in the console version, which introduced basic challenge elements to the gameplay. These sprites were designed to be replaced with graphic components once Swing was integrated.

Muslim worked on the local multiplayer logic, allowing both players to use the same keyboard to control their characters. Although simultaneous movement was not yet supported due to console limitations, the basic structure was in place for future expansion.

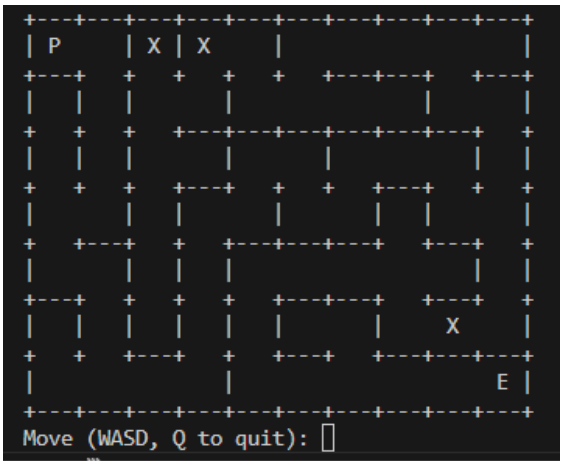


Figure 4. *Console-based maze from the previous iteration showing player, exit, and placeholder sprites.*

Made by Carlos Urra

Modifications

Mirrored Maze Generation
The maze generation algorithm was improved to support mirrored layouts. This enhancement ensures fairness by generating symmetric paths, allowing both players to navigate mazes of equal complexity from opposite starting positions.[3]

Player Start and Exit Logic
The start and exit positions for both players were defined to reinforce balanced gameplay. Player 1 begins at the top-left corner and must reach the bottom-right exit, while Player 2 starts at the bottom-right and aims for the top-left. This setup encourages parallel competition within the maze.

Improved Sprite Behavior
The sprites were enhanced to move randomly within the maze, increasing unpredictability and challenge. In addition to navigating around walls, sprites can now actively "catch" players if they land on the same cell, resulting in a game-over scenario. This adds a layer of strategy and urgency to the game.

Unit Testing with JUnit
To ensure functional reliability, a dedicated test class was developed using JUnit. These tests focus on verifying player movement, wall collision, and initial conditions, contributing to better debugging and future-proofing the application.[6]

Branch Integration and Conflict Resolution
Significant effort was dedicated to merging different development branches, each containing unique features such as sprite handling, mirrored mazes, and multiplayer logic. Conflicts were manually resolved, and a consistent, integrated version of the game was successfully established in the main branch.[7]

Game Logic Enhancements
Core gameplay logic was refined to ensure accurate exit detection and player feedback. End-game messages were adjusted to clearly indicate victory or defeat based on player actions and interactions with sprites.

Made by Carlos Urra

Keyboard Input Translation

A switch-case mapping system was introduced to support dual control schemes. Player 2's IJKL inputs are now internally translated into WASD commands, allowing both users to interact with the maze using intuitive, non-overlapping keys.[5]

Collaboration

Student 1: Diana Ali Silva

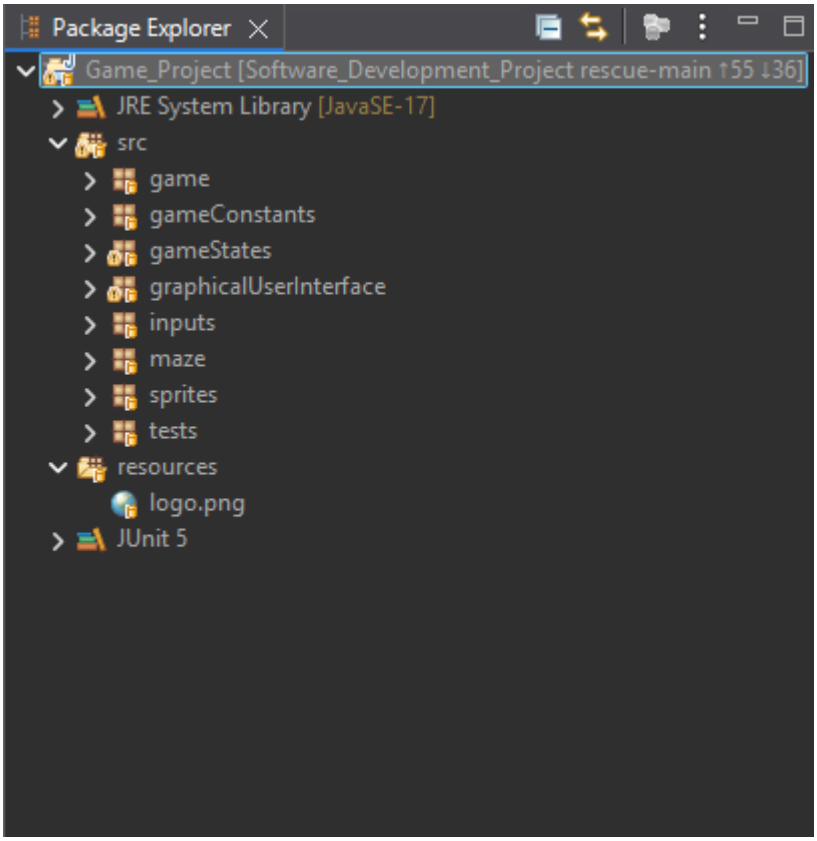
Throughout the second development of this project, I worked using Eclipse IDE, making steady progress. I have made a total of 55 commits on the rescue-main branch, reflecting a detailed and iterative development process. My contributions included critical bug fixes, feature implementations, and code refactoring aimed at improving maintainability and user experience. One of the most significant changes I made was transforming the project from a console-based game into a graphical application using Java Swing. This required reworking the existing logic and carefully integrating it with GUI components to ensure a smoother, more interactive user experience.

I thoroughly tested all changes to confirm they functioned as expected within the new Swing-based structure, and ensured they aligned with the project's goals.

Despite this progress, I have encountered an issue preventing me from pushing my local commits to the remote repository. While attempting to push through both Eclipse and Git Bash, the operation was rejected with a “non-fast-forward” error. To avoid conflicts or overwriting others’ work, I refrained from force-pushing.

To resolve this, I plan to notify my lecturer Tracey Cassels as she had helped before with Eclipse technical issues. As a precaution, I have backed up all local work and can push my commits to a new branch if necessary. While this delay is frustrating to myself and my team, the project is ready for integration once the push issue is resolved.

Diana's Screenshots



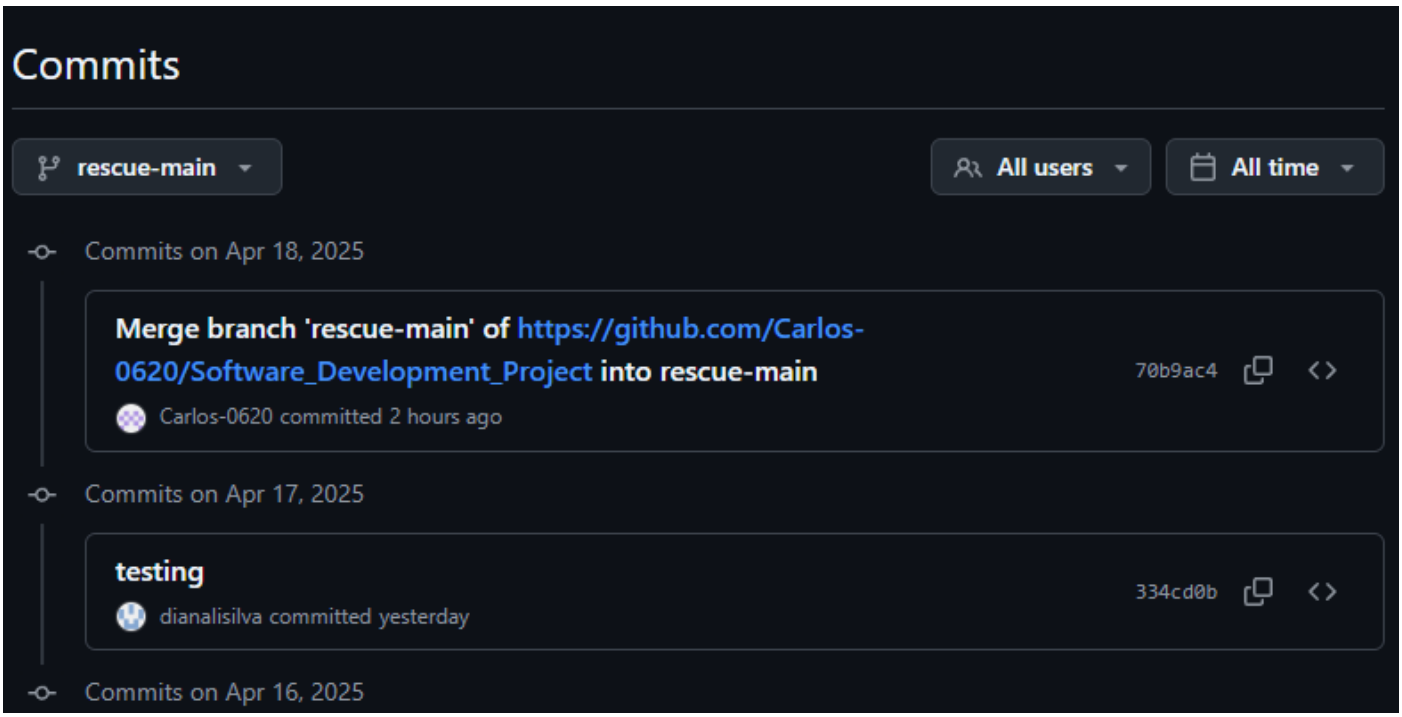
Package Explorer with packages I created for GUI implementation.

History | Synchronize | Git Staging | Git Reflog | Properties

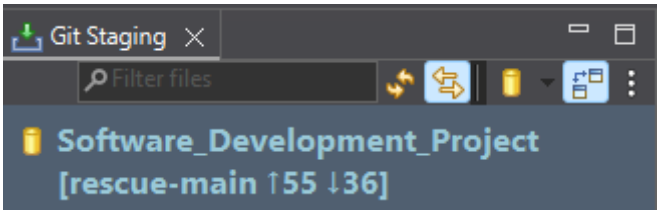
Repository: Software_Development_Project

Id	Message	Author	Authored Date	Committer	Committed Date
d68966e	rescue-main HEAD Tried to add logo to window (Unsuccessful).	Diana Silva	62 minutes ago	Diana Silva	62 minutes ago
2056e54	Made a resource folder for the logo.	Diana Silva	87 minutes ago	Diana Silva	87 minutes ago
718138e	Changed code to accept two players.	Diana Silva	2 hours ago	Diana Silva	2 hours ago
f906aa9	changed the render method to include (Graphics g, Player player).	Diana Silva	2 hours ago	Diana Silva	2 hours ago
9b0eb87	Fixed coding errors.	Diana Silva	2 hours ago	Diana Silva	2 hours ago
2609681	Fixed coding errors.	Diana Silva	2 hours ago	Diana Silva	2 hours ago
d0bc774	Added method for setRectPos.	Diana Silva	2 hours ago	Diana Silva	2 hours ago
218fc80	Added Getters and Setters.	Diana Silva	2 hours ago	Diana Silva	2 hours ago
ed3c3b7	Added missing StateMethods mathods.	Diana Silva	2 hours ago	Diana Silva	2 hours ago
513e02b	Added a missing bracket and changed coordinates for the buttons.	Diana Silva	3 hours ago	Diana Silva	3 hours ago
992f184	State entends JPanel.	Diana Silva	3 hours ago	Diana Silva	3 hours ago
5090754	Code now allows independent movement of two players.	Diana Silva	3 hours ago	Diana Silva	3 hours ago
c8a29a5	Added render method.	Diana Silva	3 hours ago	Diana Silva	3 hours ago
afbe602	Changed isAtExit() to isGoalReached().	Diana Silva	3 hours ago	Diana Silva	3 hours ago
eff967a	Added update and render methods.	Diana Silva	3 hours ago	Diana Silva	3 hours ago
89a60b6	Modified update method.	Diana Silva	3 hours ago	Diana Silva	3 hours ago
3f5db34	Added switch case to setGameState.	Diana Silva	4 hours ago	Diana Silva	4 hours ago
248ec1b	Declared State class.	Diana Silva	4 hours ago	Diana Silva	4 hours ago
376f27e	Added method to access the menu through keyboard controls.	Diana Silva	4 hours ago	Diana Silva	4 hours ago
b6fc879	Made state class abstract.	Diana Silva	4 hours ago	Diana Silva	4 hours ago
0b4e806	Added player 2 for coop implementation	Diana Silva	4 hours ago	Diana Silva	4 hours ago
b7b9af9	Added public static GameState.	Diana Silva	4 hours ago	Diana Silva	4 hours ago
606a072	Removed public static GameState state = MENU;	Diana Silva	4 hours ago	Diana Silva	4 hours ago
0059df3	Added GameState to MazeGame class.	Diana Silva	4 hours ago	Diana Silva	4 hours ago
4d36b54	Fixed coding errors.	Diana Silva	4 hours ago	Diana Silva	4 hours ago
215e7d6	Added mouse events for whenever the mouse is clicked.	Diana Silva	6 hours ago	Diana Silva	6 hours ago
c01b6e7	Removed GAME_OVER from enum.	Diana Silva	6 hours ago	Diana Silva	6 hours ago
85a8eaa	Deleted mouse and keyboard events.	Diana Silva	6 hours ago	Diana Silva	6 hours ago
ca9d8d6	Added mouse and keyboard events.	Diana Silva	6 hours ago	Diana Silva	6 hours ago
3be599e	Menu now extends State.	Diana Silva	6 hours ago	Diana Silva	6 hours ago
a481eee	Fixing code bugs.	Diana Silva	6 hours ago	Diana Silva	6 hours ago
80da650	Playing now extends State and implements StateMethods.	Diana Silva	6 hours ago	Diana Silva	6 hours ago
6ccc788	Changed game to protected instead of private.	Diana Silva	6 hours ago	Diana Silva	6 hours ago
0bb27ed	Added a getter to State.java	Diana Silva	6 hours ago	Diana Silva	6 hours ago
4d7508a	Created a superclass for all gamestates.	Diana Silva	6 hours ago	Diana Silva	6 hours ago
41a1d7f	Implemented KeyListener to Frame.java.	Diana Silva	21 hours ago	Diana Silva	21 hours ago
f49f72d	Created an inputs package.	Diana Silva	22 hours ago	Diana Silva	22 hours ago
9a532c3	Added a KeyListener.	Diana Silva	22 hours ago	Diana Silva	22 hours ago
362076f	Centred Game Window.	Diana Silva	22 hours ago	Diana Silva	22 hours ago
98f72f6	Fixed coding errors.	Diana Silva	22 hours ago	Diana Silva	22 hours ago
451d411	Added more private methods.	Diana Silva	22 hours ago	Diana Silva	22 hours ago
d6c4eff	Made changes to MazeGame and started implementing Game into window.	Diana Silva	23 hours ago	Diana Silva	23 hours ago
f66ca9f	Interface created to handle methods created	Diana Silva	23 hours ago	Diana Silva	23 hours ago
2247d4d	Altered the code to handle game states and react accordingly.	Diana Silva	23 hours ago	Diana Silva	23 hours ago
a7a3e6e	Enum Creation.	Diana Silva	23 hours ago	Diana Silva	23 hours ago
73dfac5	Created classes to track user input in the menu.	Diana Silva	23 hours ago	Diana Silva	23 hours ago
0cb2f46	"Created a new package called gameStates."	Diana Silva	23 hours ago	Diana Silva	23 hours ago
334cd0b	fix-detached-head testing	Diana Silva	26 hours ago	Diana Silva	26 hours ago

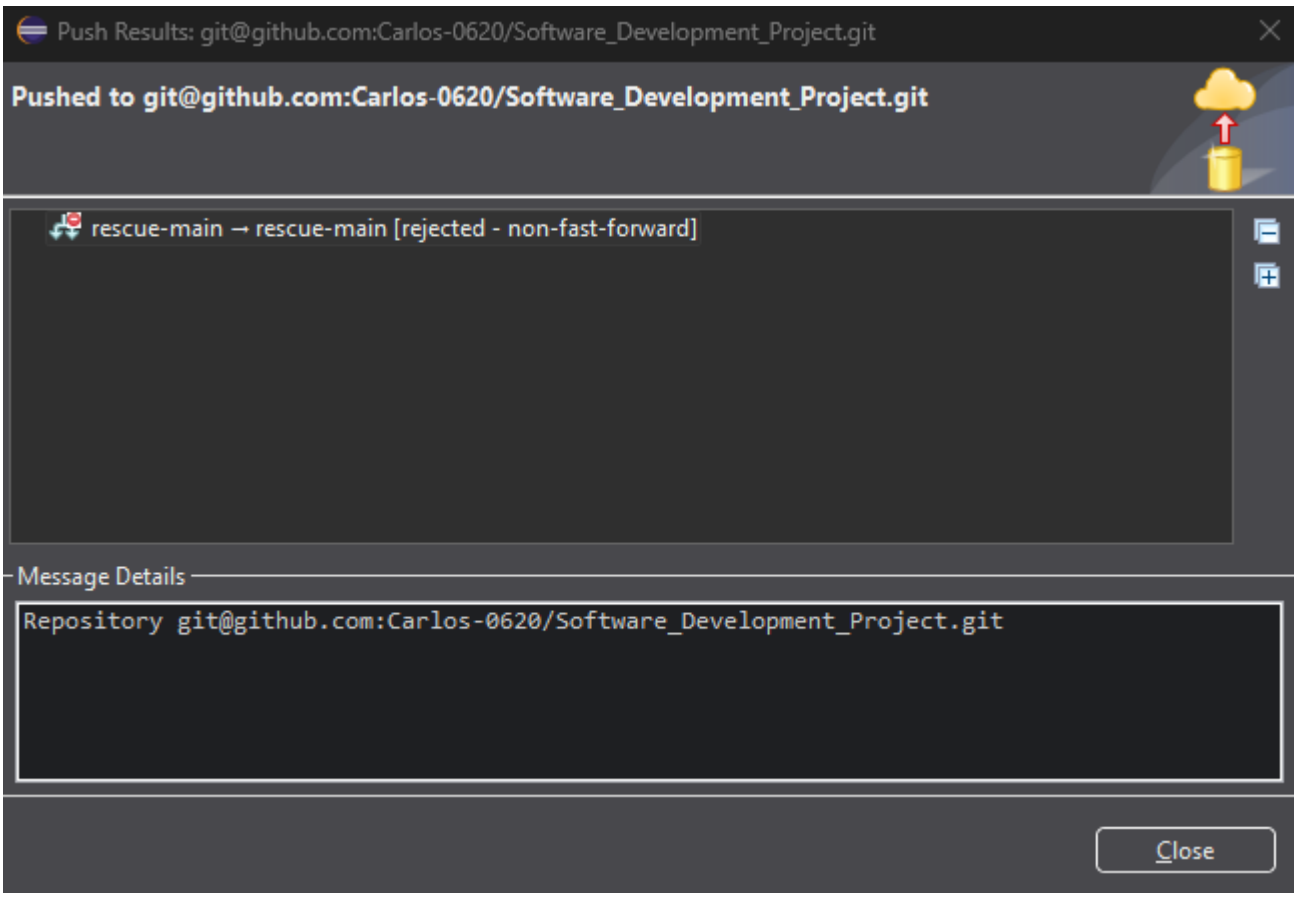
Commit history in Eclipse



Last Commit in github: Testing push with Tracey Cassels.



Commits in Eclipse IDE.



Error when pushing commits to remote repository: [rejected-non-fast-forward]

Student 2: Carlos David Urra Cabello

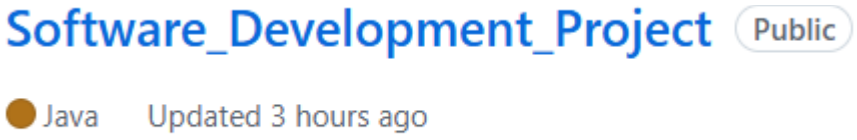
Building on the foundation from Iteration 1, where I applied the maze generation algorithm, set up the basic game logic, and organized the repository, I continued supporting the project by integrating new features and streamlining the overall structure.

In this phase, I implemented a switch-case input translation system to support multiplayer. This allowed Player 2 to use the IJKL keys, which are internally mapped to the standard WASD movement used by Player 1. The solution was based on publicly available examples and improved the game's control system for shared-keyboard gameplay.

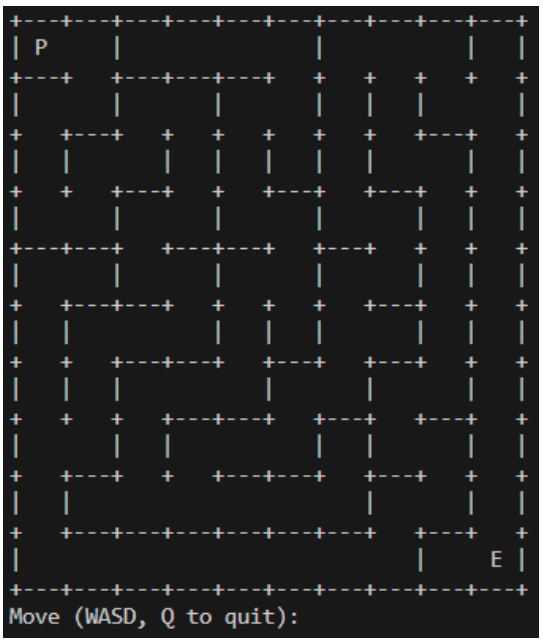
I also handled the merging of various development branches, each containing specific updates such as sprite behaviour, maze mirroring, and multiplayer adjustments. During this process, I ensured that all features were properly combined into a single, working version.

Additionally, I added a JUnit test class to check key parts of the game logic. These unit tests help confirm correct player movement, wall detection, and basic setup conditions, supporting code reliability as the game evolves.

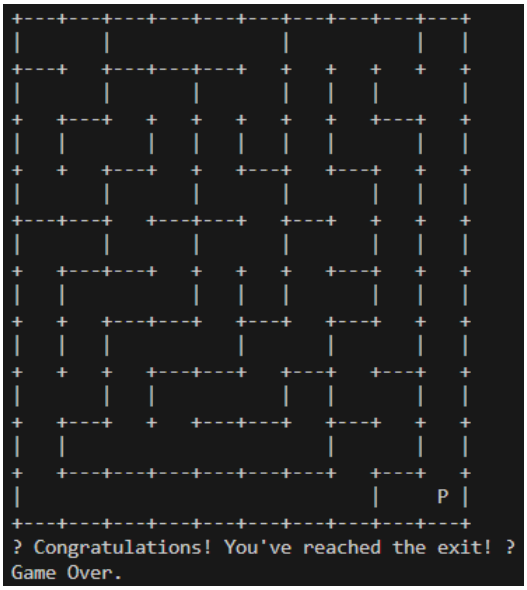
Carlos's Screenshots



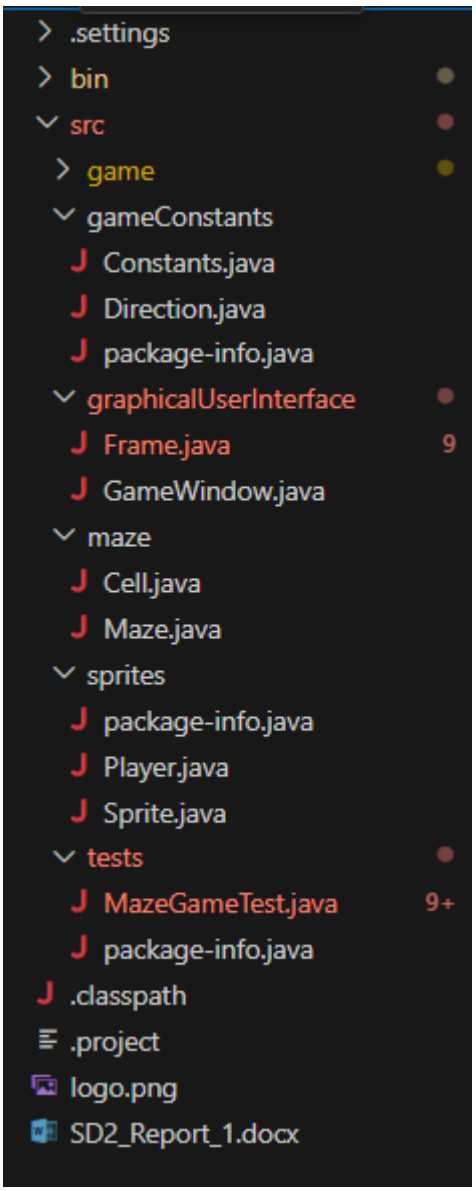
Github repository



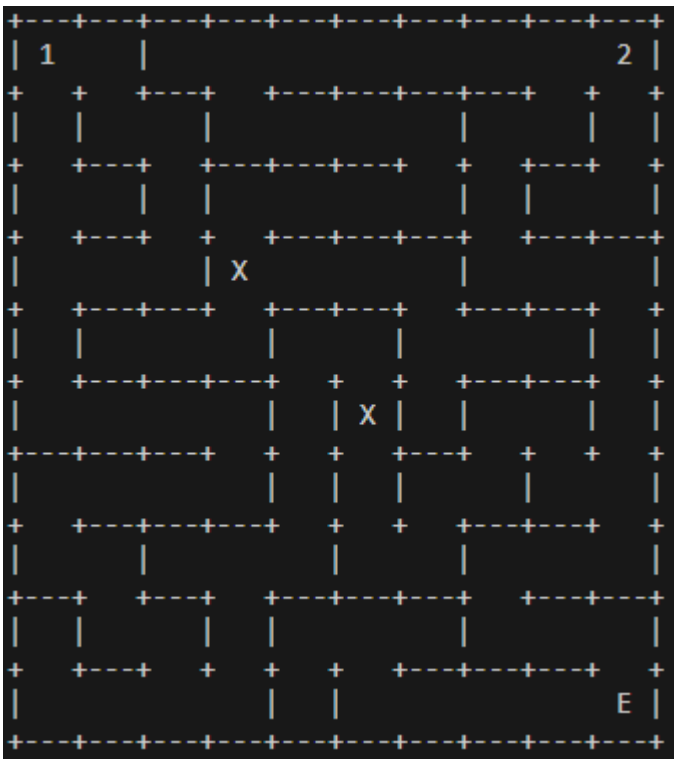
randomly generated maze



Exit message shown when the player successfully reaches the end of the maze.



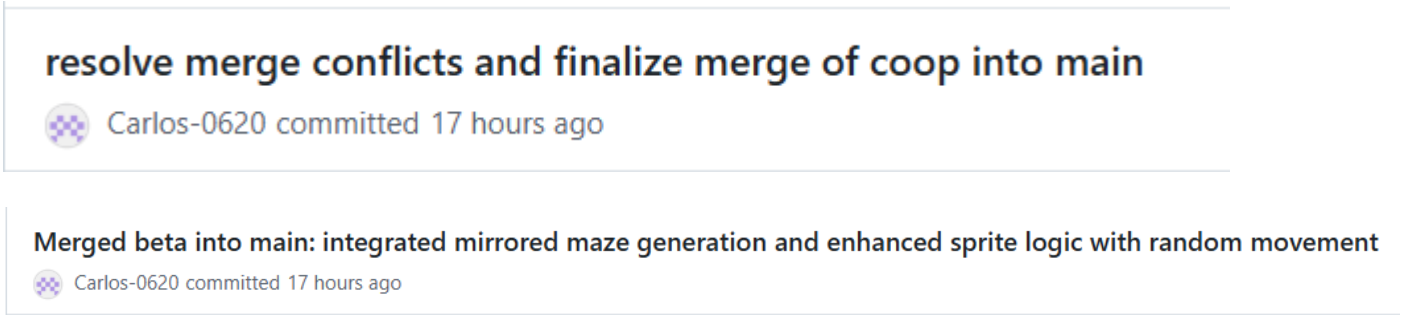
Organized folder structure based on the example provided on Moodle.



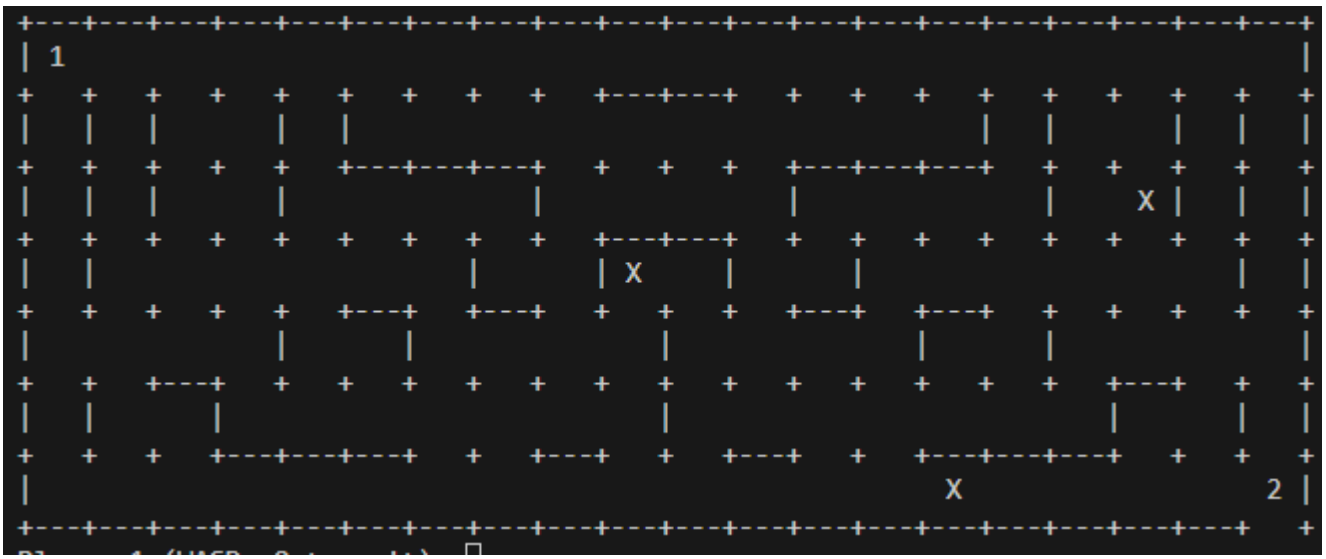
Multiplayer maze view showing Player 1 and Player 2 starting at opposite corners.

```
72          String translatedDir = switch (input) {
73              case "I" -> "W";
74              case "K" -> "S";
75              case "J" -> "A";
76              case "L" -> "D";
77              default -> "";
78          };
79
```

Java switch expression used to translate IJKL inputs into WASD equivalents for Player 2 movement, enabling intuitive local multiplayer control.



Commits showing Carlos's work on merging branches and integrating key gameplay features.



Player 1 starts in the top-left corner and Player 2 in the bottom-right.

Problems Encountered

Multiplayer Input Issues

One of the first challenges was getting Player 2's movement to work properly. Although input mapping using a switch statement was set up, the second player's commands were not reflected in the maze. This was due to mishandling of input translation and inconsistent updates to player positions during the game loop.[5]

Maze Rendering Conflicts

At one point, the maze failed to display the players correctly on the console. This happened because the logic responsible for placing characters in the printed maze did not account for both players occupying unique positions. Updating the print method to handle overlapping entities solved this issue.

Merge Conflicts

When merging the coop and beta branches into main, several merge conflicts occurred, especially in MazeGame.java and Maze.java. These had to be resolved manually to preserve new features from both branches while avoiding duplication or logic errors.[7]

Unit Testing Difficulties

Testing player movement and wall collision proved tricky due to the randomness of maze generation. Some tests would occasionally fail when a randomly generated wall blocked an expected movement. The solution was to adjust the tests to conditionally check valid directions before asserting position changes.[6]

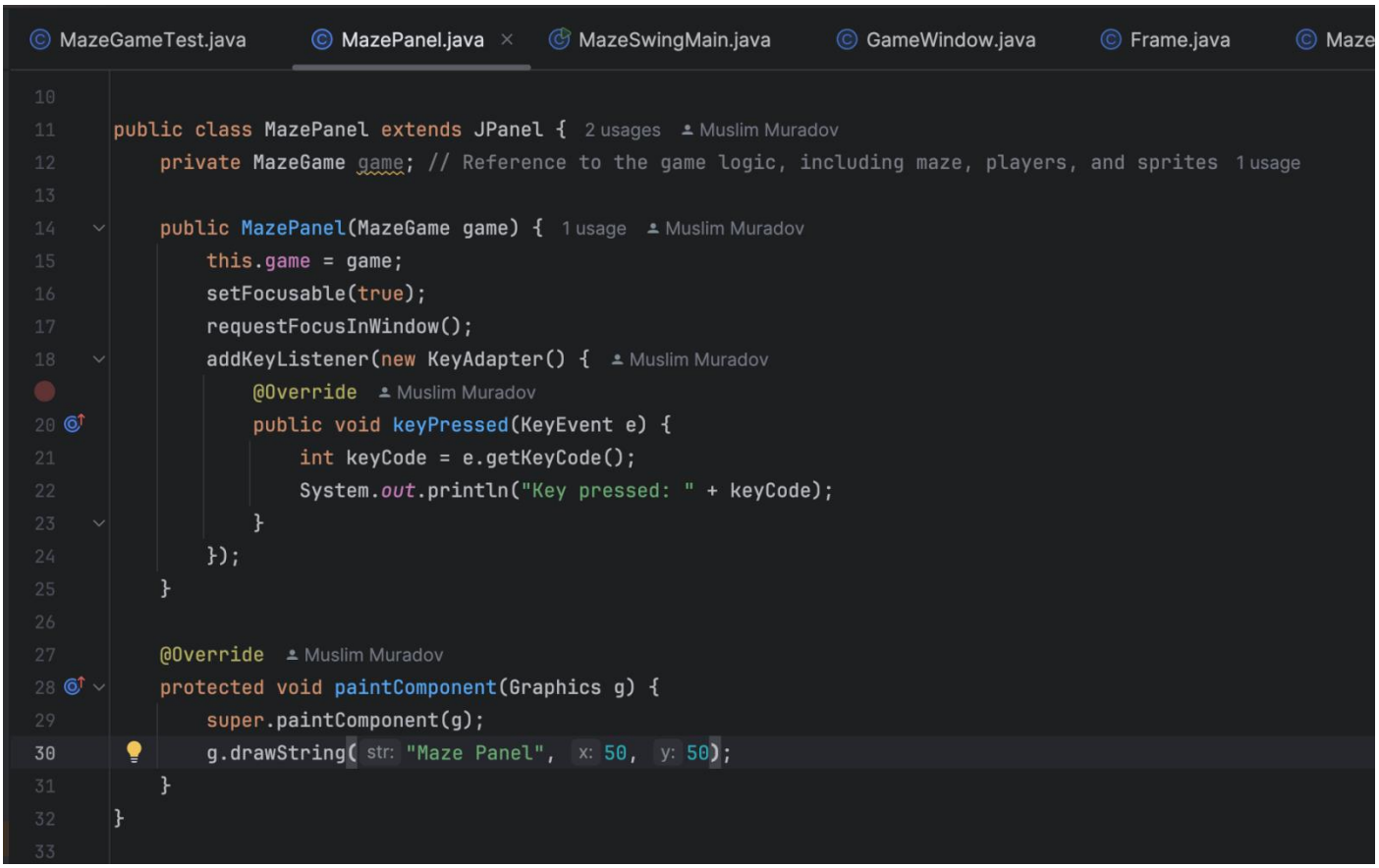
Student 4: Muslim Muradov

In this part of the work, I integrated some graphical user interface (GUI) components into a maze game using Java Swing. This involved creating a MazePanel class to display a window and handling keyboard input to move the player. For now, I've closed this with placeholder so that I can later implent the maze itself and the players into the game window

Running the Swing application was difficult at first, I had to create a separate main class to run it. I kept the previous class so that the game could still be run in the console and tested. When we move the maze to Swing, we can get rid of the console version.

All in all, these additions laid the groundwork for further development of the game in a convenient window to ensure convenient gameplay and proper operation.

Muslim's Screenshots



MazeGameTest.java | MazePanel.java | MazeSwingMain.java | GameWindow.java

```java
package graphicalUserInterface;

import game.MazeGame;

//MazeSwingMain is a test class to launch the Swing version of the maze game.
public class MazeSwingMain {  Muslim Muradov
    public static void main(String[] args) {  Muslim Muradov
        // Create an instance of MazeGame
        MazeGame game = new MazeGame();

        // Create the main game frame passing the game logic
        Frame frame = new Frame(game);

        // Set the frame visible to start the GUI
        frame.setVisible(true);
    }
}
```

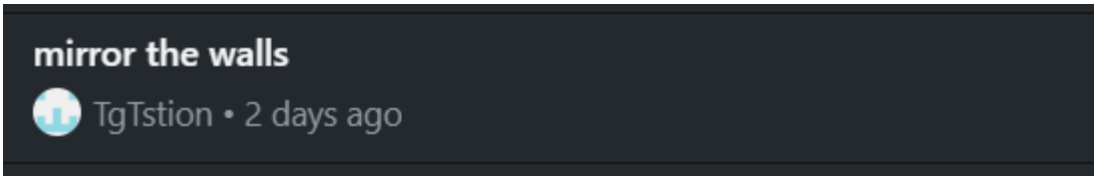
Added some changes in Frame class

```java
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setResizable(true);
setSize( width: 1080,  height: 720);
setLocationRelativeTo(null);
setVisible(true);

initializeGameUI();

MazePanel mazePanel = new MazePanel(game);
add(mazePanel);
```

Student 3: Georgii Taisaev

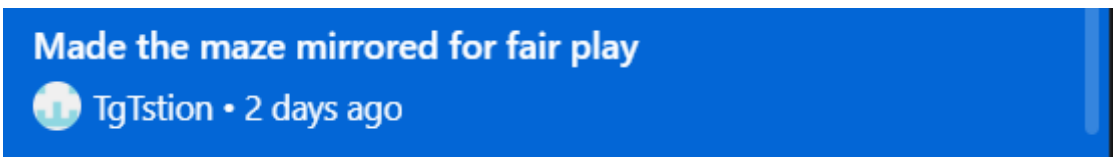
Georgii's Screenshots

1) made the walls in the maze reflective

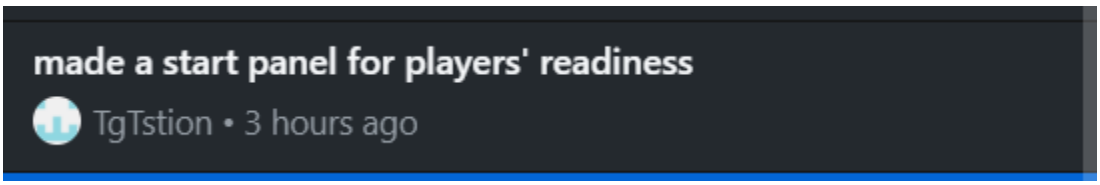



```
        for (int row = 0; row < height; row++) {
            if (width % 2 != 0) width++;
            for (int col = 0; col < width; col++) {
                grid[row][col] = new Cell(row, col);
            }
        }

        this.exitRow = height - 1;
        this.exitCol = width - 1;
    }

    public void generateMaze() {
        Stack<Cell> stack = new Stack<>();
        Cell start = grid[0][0];
        start.visited = true;
        stack.push(start);
```

2) I made the maze perfectly symmetrical on both sides to ensure fairness for both players



3) created a player readiness panel in Java Swing for transitioning to the main game


made a start panel for players' readiness
TgTstion • 3 hours ago

```
        cardLayout = new CardLayout();
        cardPanel = new JPanel(cardLayout);


        readyPanel = new ReadyPanel(() -> startGame(game));
        cardPanel.add(readyPanel, constraints:"Ready");


        mazePanel = new MazePanel(game);
        cardPanel.add(mazePanel, constraints:"Game");

        add(cardPanel);
        setVisible(b:true);


        cardLayout.show(cardPanel, name:"Ready");
    }
    private void startGame(MazeGame game) {

        mazePanel.requestFocusInWindow();
        cardLayout.show(cardPanel, name:"Game");
    }
    private void initializeGameUI() {

    }
    public void setBackgroundColor(Color color) {
        getContentPane().setBackground(color);
    }
```

```java
public class Frame extends JFrame {
    private CardLayout cardLayout;
    private JPanel cardPanel;
    private ReadyPanel readyPanel;
    private MazePanel mazePanel;

    public Frame(MazeGame game){
        super(title:"game"); //Calls for parents constructor with game reference,

        //Customisation of window settings
        ImageIcon logo = new ImageIcon(filename:"logo.png"); //Creates an ImageIcon.
        setIconImage(logo.getImage()); //Changes icon of frame.
        getContentPane().setBackground(new Color(r:64, g:64, b:64)); //Changes colour of the background.
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(resizable:false);
        setSize(width:1080, height:720);
        setLocationRelativeTo(c:null);
        setVisible(b:true);

        initializeGameUI();


        cardLayout = new CardLayout();
        cardPanel = new JPanel(cardLayout);


        readyPanel = new ReadyPanel(() -> startGame(game));
        cardPanel.add(readyPanel, constraints:"Ready");


        mazePanel = new MazePanel(game);
        cardPanel.add(mazePanel, constraints:"Game");

        add(cardPanel);
        setVisible(b:true);
```

```java
public void generateMaze() {
    Stack<Cell> stack = new Stack<>();
    Cell start = grid[0][0];
    start.visited = true;
    stack.push(start);

    while (!stack.isEmpty()) {
        Cell current = stack.peek();
        Cell next = getRandomUnvisitedNeighbor(current);

        if (next != null) {
            removeWalls(current, next);
            next.visited = true;
            stack.push(next);
            int mirroredRow = current.row;
            int mirroredCol = width - 1 - current.col;
            int mirroredNextRow = next.row;
            int mirroredNextCol = width - 1 - next.col;
            removeWalls(grid[mirroredRow][mirroredCol], grid[mirroredNextRow][mirroredNextCol]);
        } else {
            stack.pop();
        }
    }
}

private Cell getRandomUnvisitedNeighbor(Cell cell) {
```

4) created a button that transitions to another visual window with the further gameplay of our game

start game button

TgTstion • 3 hours ago

```
private void startGame(MazeGame game) {

    mazePanel.requestFocusInWindow();
    cardLayout.show(cardPanel, name:"Game");
}
```

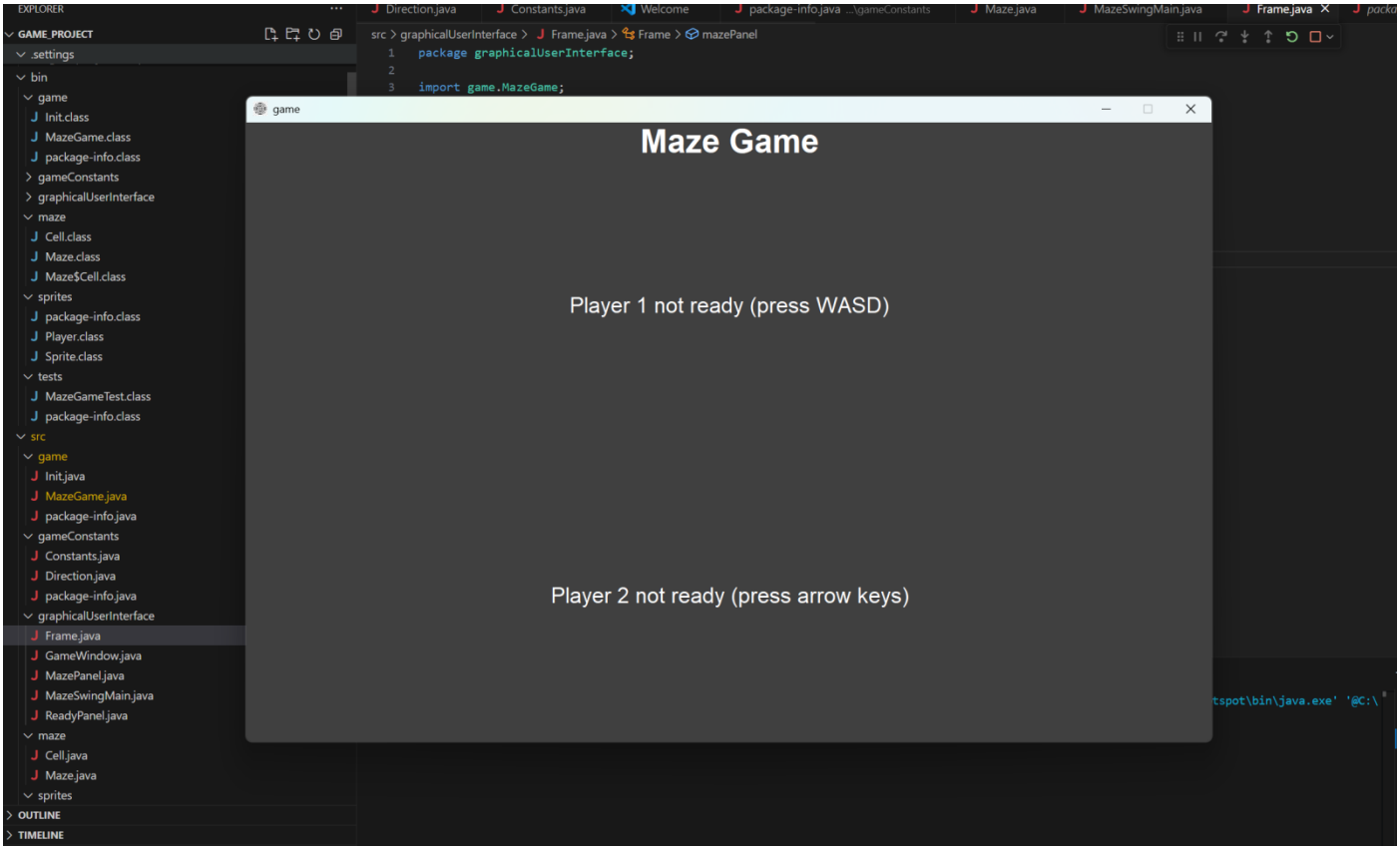
5) I made it so that pressing the WASD keys makes one player ready, while pressing the arrow keys makes the other player ready

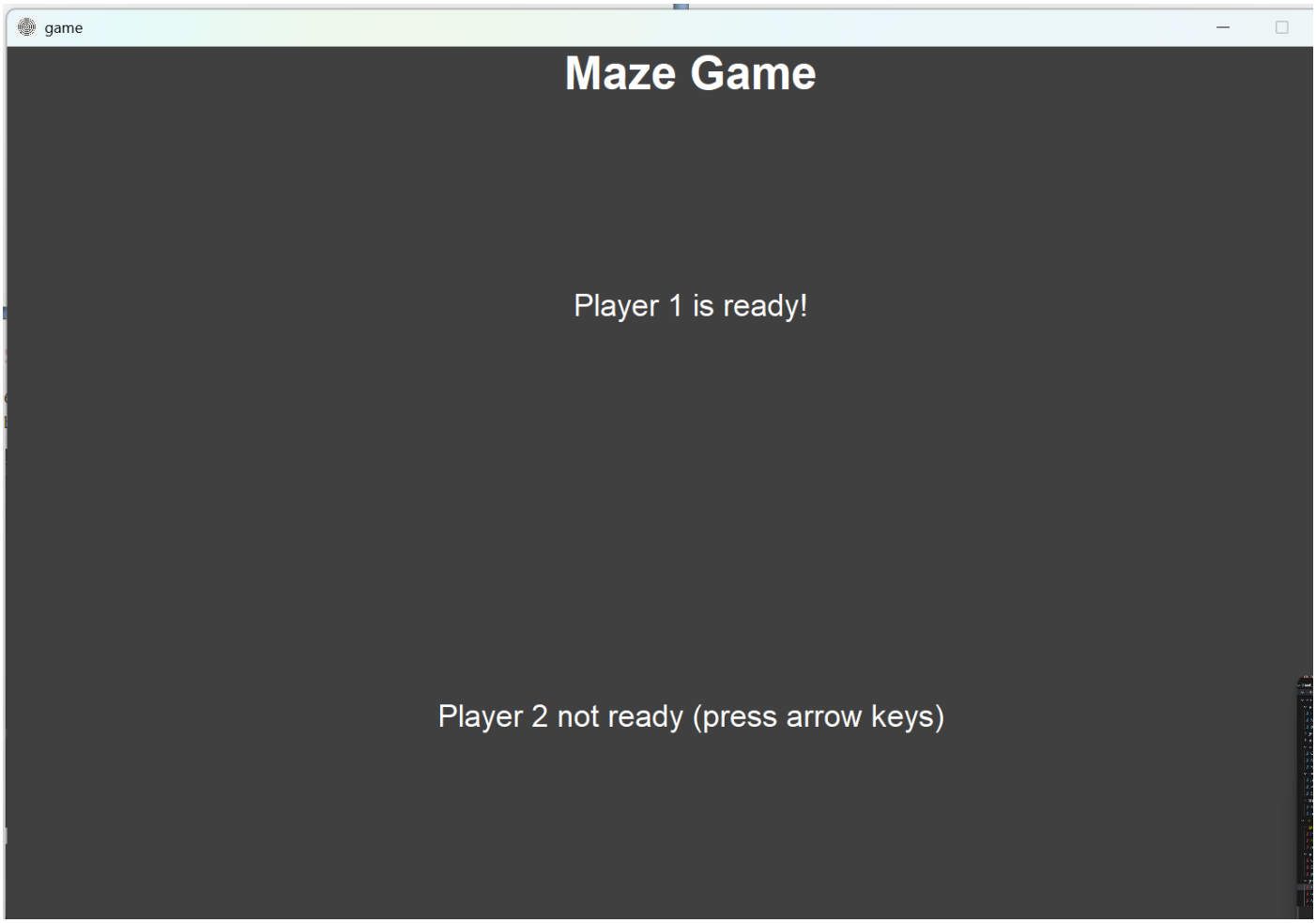
Fill the ready, panel and switch to the main game terminal

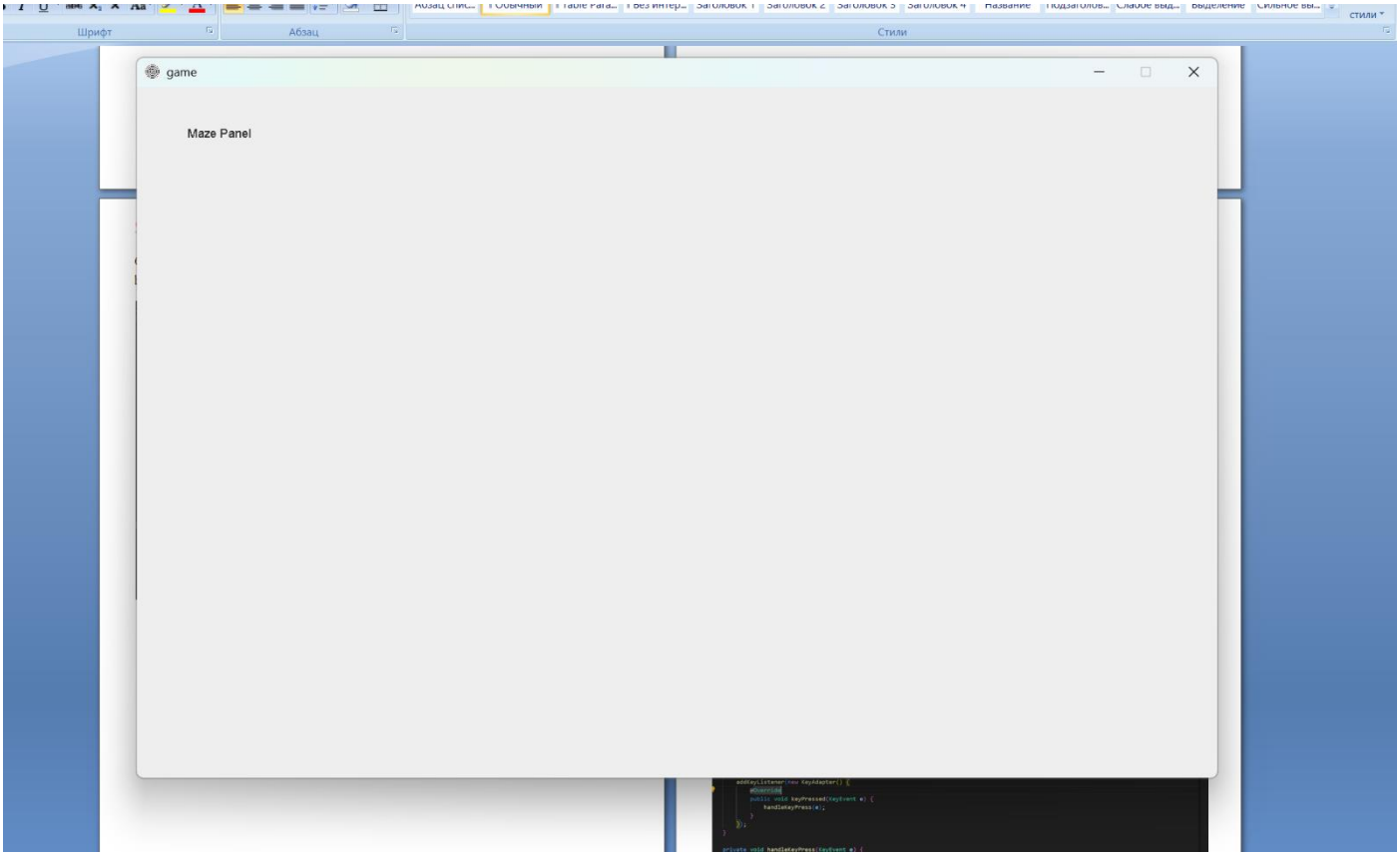
TgTstion • 3 hours ago

```
package graphicalUserInterface;

import javax.swing.*;
import java.awt.*;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;

public class ReadyPanel extends JPanel {
    private boolean player1Ready = false;
    private boolean player2Ready = false;
    private JLabel player1Status;
    private JLabel player2Status;
    private Runnable onBothReady;

    public ReadyPanel(Runnable onBothReady) {
        this.onBothReady = onBothReady;

        setLayout(new BorderLayout());
        setBackground(Color.DARK_GRAY);

        JLabel title = new JLabel(text:"Maze Game", SwingConstants.CENTER);
        title.setFont(new Font(name:"Arial", Font.BOLD, size:36));
        title.setForeground(Color.WHITE);
        add(title, BorderLayout.NORTH);

        JPanel statusPanel = new JPanel(new GridLayout(rows:2, cols:1));
        statusPanel.setBackground(Color.DARK_GRAY);

        player1Status = new JLabel(text:"Player 1 not ready (press WASD)", SwingConstants.CENTER);
        player1Status.setFont(new Font(name:"Arial", Font.PLAIN, size:24));
        player1Status.setForeground(Color.WHITE);

        player2Status = new JLabel(text:"Player 2 not ready (press arrow keys)", SwingConstants.CENTER);
        player2Status.setFont(new Font(name:"Arial", Font.PLAIN, size:24));
        player2Status.setForeground(Color.WHITE);
```

6) «I completely removed the start button; now the game starts automatically when both players are ready




game
Maze Game
Player 1 is ready!
Player 2 not ready (press arrow keys)


game
Maze Panel

Replaced the start button with automatic readiness and progression once players press WASD or arrow keys

TgTstion • 3 hours ago

```
    player2Status = new JLabel(text:"Player 2 not ready (press arrow keys)", SwingConstants.CENTER);
    player2Status.setFont(new Font(name:"Arial", Font.PLAIN, size:24));
    player2Status.setForeground(Color.WHITE);

    statusPanel.add(player1Status);
    statusPanel.add(player2Status);
    add(statusPanel, BorderLayout.CENTER);


    setFocusable(focusable:true);
    requestFocusInWindow();


    addKeyListener(new KeyAdapter() {
        @Override
        public void keyPressed(KeyEvent e) {
            handleKeyPress(e);
        }
    });
}

private void handleKeyPress(KeyEvent e) {
    int keyCode = e.getKeyCode();
    if (!player1Ready && (keyCode == KeyEvent.VK_W || keyCode == KeyEvent.VK_A || keyCode == KeyEvent.VK_S || keyCode == KeyEvent.VK_D)) {
        player1Ready = true;
        player1Status.setText(text:"Player 1 is ready!");
        checkReadyStatus();
    } else if (!player2Ready && (keyCode == KeyEvent.VK_UP || keyCode == KeyEvent.VK_DOWN || keyCode == KeyEvent.VK_LEFT || keyCode == KeyE
        player2Ready = true;
        player2Status.setText(text:"Player 2 is ready!");
        checkReadyStatus();
    }
```

7) I fixed a bug where the program wouldn't respond to keyboard inputs when the window was opened, preventing players from getting ready. I adjusted the process hierarchy to resolve it

Fixed a bug where the focus was not always on the window for button presses to mark the player as ready

TgTstion • 3 hours ago

```
    @Override
    public void addNotify() {
        super.addNotify();
        requestFocusInWindow();
    }
}
```

Planned Improvements for the final version

he final version of the project is expected to deliver a fully functional graphical user interface (GUI) using Java Swing, replacing the previous console-based interaction. This enhancement will significantly improve the overall user experience by providing a more intuitive and visually engaging way to interact with the game. Rather than relying on ASCII characters and text-based prompts, the game will feature a visual rendering of the maze, player avatars, and moving sprites, offering a more modern and appealing interface.

One of the most important improvements will be enabling simultaneous multiplayer functionality. In the current version, both players take turns entering commands in the console. With the GUI, the goal is to support real-time keyboard input for both players, allowing them to move at the same time. This change will enhance gameplay fluidity and bring the experience closer to traditional cooperative or competitive games.

The visual design of the game will also be updated by introducing graphical representations for the sprites. Instead of using simple 'X' characters, sprites will be shown as unique images or icons on the screen. This will make the game visually clearer and more enjoyable, especially when sprites interact with players, such as during collisions or when players are caught.

Another planned improvement is the addition of a basic main menu. This menu would allow players to start a new game, configure maze dimensions, or exit the application. Including customizable settings such as difficulty level or sprite behaviour will give users more control over their experience and accommodate different skill levels.

Finally, the game will integrate visual and possibly audio feedback to better communicate events to the player. For instance, when a collision occurs, the game could highlight the affected cell or trigger a sound effect to make the event more noticeable. These changes are intended to create a polished, user-friendly, and engaging game environment in the final version.

Made by Carlos Urra

Link to our repository:
https://github.com/Carlos-0620/Software_Development_Project.git

References

1. Midway Manufacturing Co. (1976). Amazing Maze [Arcade Game]. International Arcade Museum. Retrieved from https://www.arcade-museum.com/Videogame/amazing-maze
2. Atari. (1980). Maze Craze [Atari 2600 Game]. Atari Official Website. Retrieved from https://atari.com/pages/mazecraze
3. Buck, J. (2015). Mazes for Programmers: Code Your Own Twisty Little Passages. The Pragmatic Bookshelf. ISBN: 978-1-68050-055-4
4. GeeksforGeeks. (n.d.). Introduction to Java Swing. Retrieved from https://www.geeksforgeeks.org/introduction-to-java-swing/
5. GeeksforGeeks. (2022). Switch statement in Java. Retrieved from https://www.geeksforgeeks.org/switch-statement-in-java/
6. JUnit Team. (n.d.). JUnit 5 User Guide. Retrieved from https://junit.org/junit5/docs/current/user-guide/
7. GitHub Docs. (n.d.). About merge conflicts. Retrieved from https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/about-merge-conflicts